

0. Idea

Un **DataFrame** es una tabla: columnas con nombre (cada una un array de numpy con su tipo) + un **índice** de filas. Ventaja frente a numpy: seleccionas por *nombre* y por *valor*, no por posición.

1. Crear / leer

```
# de una lista de dicts: cada dict = una fila,
# las claves son las columnas
rows = [{'name': 'tilt_0', 'q': 1, 'I': 0.1},
        {'name': 'tilt_0', 'q': 3, 'I': 1e-3}]
harm = pd.DataFrame(rows)
```

```
# de un dict de listas: clave = columna
df = pd.DataFrame({'name': ['a','b'], 'x': [1, 2]})
```

```
# fichero separado por tabuladores, ignora '#'
sweep = pd.read_csv(path, sep='\t', comment='#')
```

Tablas del notebook:

sweep: index, name, Field.chi (una fila por cálculo)

harm: name, Field.chi, q, I, S1, S2, S3, P, chi, eps, phi, delta_varphi (una fila por cálculo × armónico)

2. Mirar la tabla

```
harm.head()      # 5 primeras filas (tail(): últimas)
harm.shape       # (n_filas, n_columnas)
harm.columns     # nombres de columnas
harm.dtypes      # tipo de cada columna
harm.describe()  # media, std, min, max... por columna
harm['name'].unique() # valores distintos
```

3. Índice de filas ≠ columna 'index'

```
sweep.index      # RangeIndex 0,1,2,... (etiquetas)
sweep['index']    # columna leída del fichero
```

Coinciden por casualidad al leer la tabla. Tras filtrar, el índice **conserva las etiquetas originales** (p.ej. 2, 5, 8), no se renumera. Para reenumerar: g.reset_index(drop=True).

4. Columnas

```
harm['I']         # una columna → Series
harm[['q', 'I']]  # varias → DataFrame
harm.q           # atajo, solo si el nombre
                 # es un identificador válido
harm['Field.chi'] # con '.' → siempre corchetes
harm['I'].to_numpy() # → array de numpy
harm['chi_deg'] = np.rad2deg(harm['chi']) # nueva
```

Las operaciones entre columnas son elemento a elemento, como en numpy.

5. Filtrar filas con una máscara booleana

```
m = harm['q'] == 5      # Series de True/False
harm[m]                # filas donde m es True
harm[harm['q'] == 5]    # lo mismo en una línea
```

```
# varias condiciones: & (y), | (o), ~ (no)
# ¡paréntesis obligatorios!
harm[(harm['q'] == 5) & (harm['I'] > 1e-6)]
harm[(harm['q'] < 5) | (harm['q'] > 15)]
harm[~(harm['q'] == 1)]
```

```
harm[harm['q'].isin([3, 5, 7])] # q en una lista
harm[harm['name'] == 'tilt_2']
```

Usa &, |, ~, no and, or, not (estos piden un único True/False y dan ValueError).

```
# alternativa textual (nombres raros entre ``)
harm.query('q == 5 and I > 1e-6')
harm.query('`Field.chi` > 0.5')
```

6. loc (etiquetas) e iloc (posiciones)

```
harm.loc[harm['q'] == 5, ['name', 'eps']]
# filas por máscara/etiqueta, cols por nombre
harm.loc[harm['q'] == 5, 'eps'] = 0.0 # asignar
harm.iloc[0] # primera fila (posición)
harm.iloc[0:2, :3] # como en numpy
```

Para *modificar* un subconjunto usa loc[máscara, col] = ...; harm[m]['eps'] = 0 modifica una copia (SettingWithCopyWarning).

7. Ordenar

```
harm.sort_values('Field.chi') # ascendente
harm.sort_values('I', ascending=False)
harm.sort_values(['name', 'q']) # 2 claves
```

Hace falta cuando: (i) plot une puntos en orden de fila y el barrido no es monótono; (ii) np.unwrap, np.diff, np.gradient, que comparan cada punto con el anterior. Devuelve una tabla nueva: g = g.sort_values(...).

8. Recorrer filas

```
for i, r in sweep.iterrows(): # i: etiqueta,
    print(r['name'], r['Field.chi']) # r: Series
```

Lento; bien para pocas filas (una por cálculo). Para operar con columnas enteras, mejor vectorizar.

Ojo: i es la *etiqueta*, no la posición. En harmonic_table, S0(win, i) funciona porque sweep tiene índice 0,1,2,...; si sweep viniera filtrado, usa sweep.reset_index(drop=True) o

```
for i, (_, r) in enumerate(sweep.iterrows()).
```

9. groupby: partir en subtablas

```
for q, g in harm.groupby('q'):
    # q: valor de la clave, g: subtabla con ese q
    # (= harm[harm['q'] == q])
    ax.plot(g['Field.chi'], g['eps'], 'o-',
            label=f'q={q}')
```

```
# sort=False: grupos en orden de aparición
# (así coinciden con la lista labels)
for (name, g), lab in zip(harm.groupby('name',
                                     sort=False), labels): ...
```

Agregar: una fila por grupo

```
harm.groupby('q')['I'].mean() # Series: q → media
harm.groupby('name')['I'].max()
harm.groupby('q').agg(I=('I', 'sum'), P=('P', 'mean'))
```

transform: mismo tamaño que la tabla (el resultado del grupo se repite en cada fila)

```
# intensidad relativa al máximo de su cálculo
harm['I_rel'] = harm['I'] / \
    harm.groupby('name')['I'].transform('max')
```

10. pivot: de tabla larga a matriz

```
E = harm.pivot(index='q', columns='name',
                values='eps')
```

```
# name tilt_0 tilt_1 tilt_2
# q
# 1      0.00   0.01   0.02
# 3      0.00   0.03   0.06
E.to_numpy() # matriz (n_q, n_calc)
```

Cada par (index, columns) debe ser único; si no, pivot_table(..., aggfunc='mean').

11. Combinar tablas

```
pd.merge(harm, sweep, on='name') # añade columnas
                                # de sweep por name
pd.concat([df1, df2], ignore_index=True) # apilar
```

12. Guardar

```
harm.to_csv('harm.txt', sep='\t', index=False,
            float_format='%.6e')
```

index=False: no escribe la columna de etiquetas de fila.

Errores típicos

- KeyError: 'x' → la columna no existe: mira df.columns.
- and/or en máscaras → ValueError; usa &, | con paréntesis.
- Métodos (sort_values, reset_index, ...) devuelven copia: reasigna.
- df.q falla con nombres como Field.chi: usa df['Field.chi'].